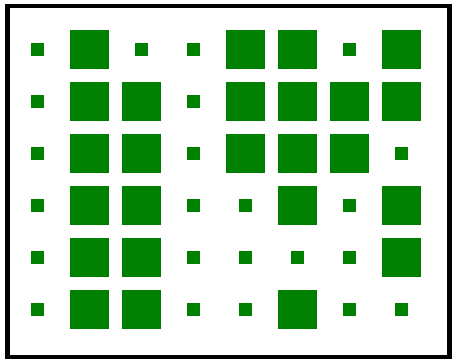
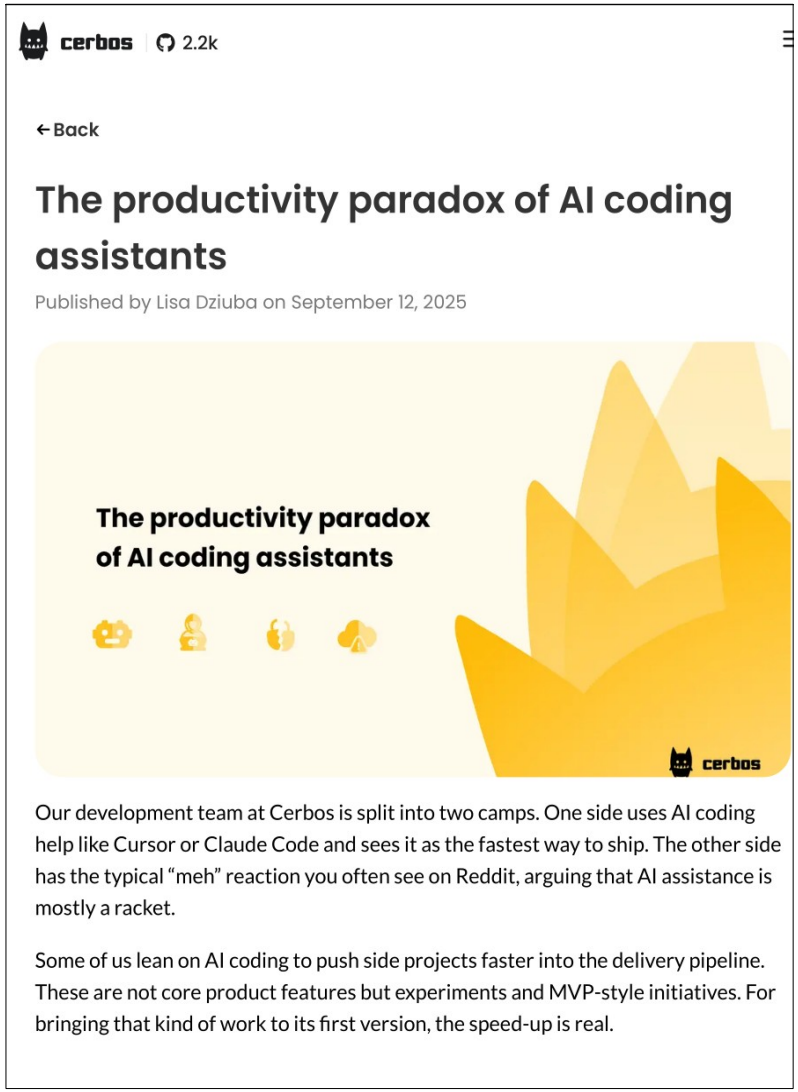


Class 9
Describing Syntax: Backus-Naur (BNF)

CS 330
Programming Languages
And Compilers



cerbos 2.2k

←Back

The productivity paradox of AI coding assistants

Published by Lisa Dziuba on September 12, 2025

The productivity paradox of AI coding assistants

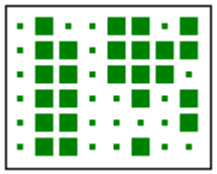
cerbos

Our development team at Cerbos is split into two camps. One side uses AI coding help like Cursor or Claude Code and sees it as the fastest way to ship. The other side has the typical “meh” reaction you often see on Reddit, arguing that AI assistance is mostly a racket.

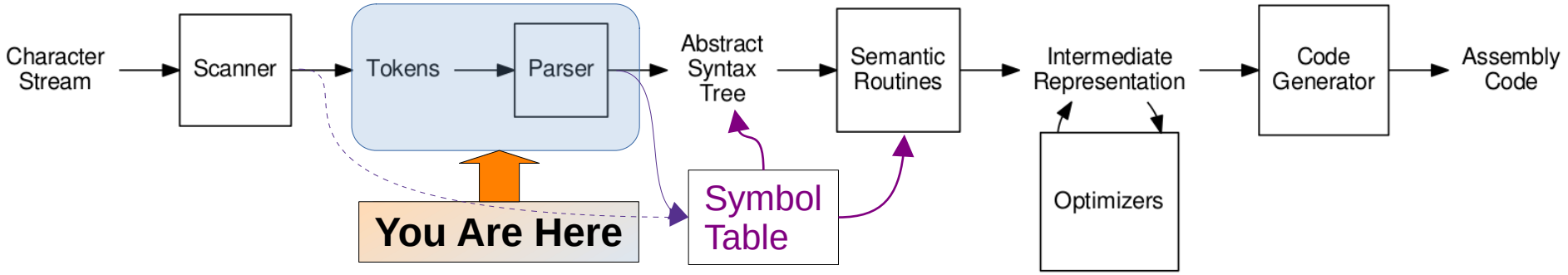
Some of us lean on AI coding to push side projects faster into the delivery pipeline. These are not core product features but experiments and MVP-style initiatives. For bringing that kind of work to its first version, the speed-up is real.

LLM Roundtable

Compilation Process Overview

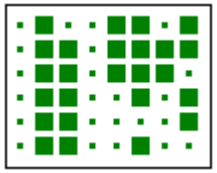


Thain, D. (2020). Introduction to compilers and language design.



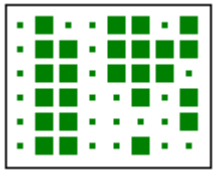
- Scanner (Lexical analyzer) – tokenizes
- **Parser (Syntax analyzer) – lexical units syntax check**
  - Creates symbol table and AST
- Semantic Routines – static semantics analysis/check
- Intermediate Rep – lower level language (optional)
- Optimizer – performance improvements (optional)
- Code Generator – outputs assembly lang (in our case)

Syntax and Semantics 1/2



- Language study breaks into two concepts
  1) Syntax
     - The rules (form) defining statements, expressions, and program units (blocks, functions, classes...)
  2) Semantics
     - The meaning of expressions, statements, and program units
- Considering natural language, can you come up with examples of these concepts?

Syntax and Semantics 2/2

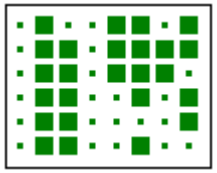


- English language example:

  Can you make me a sandwich?

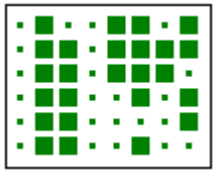
  - Syntactically correct but has multiple meanings (semantics)
- Easier to formally describe syntax than to formally describe semantics

Why Describe the Syntax?



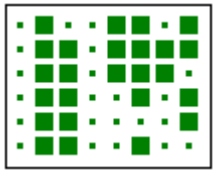
- What challenges do you see with defining a syntax?
- Who needs a formal definition of the syntax?
  - Evaluators – look for flaws in the language design
  - Implementors – create interpreters or compilers for the language
  - Users – write programs using the language

Defining a Programming Language



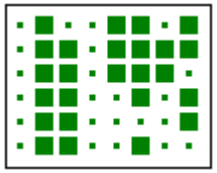
- To implement a programming language, its syntax must be very formally defined
  - Much more so than natural languages – Why?
  - Note that we are focused on the syntax
    - We'll know if the program is structurally correct, *but not what it should do*
- Two approaches to defining the syntax
  1) Language Recognizer
  2) Language Generator

Language Recognizer

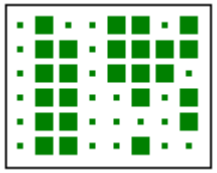


- A language recognizer defines a language by filtering legal from illegal syntax
  - Obviously not a practical way to define the language: we have to try every possible form of input (essentially infinite)
- Language recognizers perform an initial check to ensure the program is syntactically correct
  - Doesn’t mean the program is logically constructed or would “work”
- Known as a **parser**

Language Generator



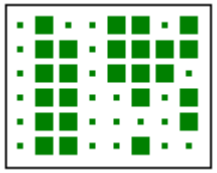
- Creates legal syntactic statements
- Would not result in working programs
  - It would generate random statements
- The generator's rules are easier to understand versus the rules for a recognizer
  - We aren't interested in creating random statements
  - We use the generator's rules to learn the language's syntax rules



Describing Syntax: Grammars

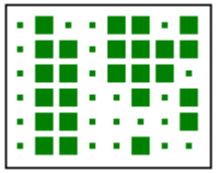
(a step toward language generation)

Context-free Grammars



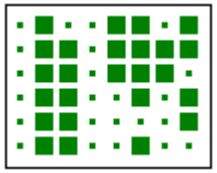
- The term **grammar** is used to describe the syntax of a programming language
  - Similar to its use when describing natural language – *rules of grammar*

Context-free Grammars



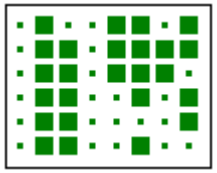
- Noam Chomsky (linguist) and John Backus (computer scientist) focused on similar problems in two different domains (natural languages and programming languages)
  - Came to independent, yet similar, approaches for describing language grammar
    - Chomsky's **regular** and **context-free grammars**
    - Backus' formal notation for specifying programming language syntax (Algol 58)

Backus-Naur Form (BNF) 1/2



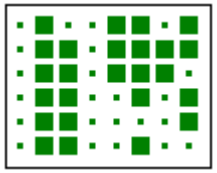
- Peter Naur modified Backus' descriptive language resulting in Backus-Naur Form (BNF)
  - Popular way to define the syntax of programming languages
  - Closely parallels Chomsky's *context-free grammars*
    - An aside, natural language processing benefits from context-free grammars

Backus-Naur Form (BNF) 2/2



- Languages, used to describe the syntax of another language, are called **metalanguages**
  - BNF is a metalanguage for describing programming languages

BNF Basics: **Abstraction**



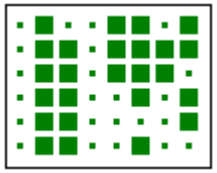
- **Abstractions** are used for syntactic structures

  `<assign>` could be used to denote an assignment statement

  Angle brackets denote abstractions

  - Name of the abstraction should help readers understand what it represents

BNF Basics: **Definition** (a.k.a. **Rule**)



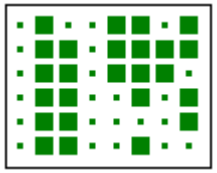
- **Definitions** show syntax of an abstraction

  `<assign>` → `<var> = <expression>`

  Abstraction (LHS) — Definition (RHS)

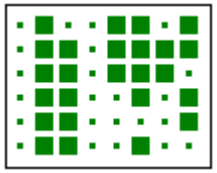
  - Left-hand side (LHS) names the abstraction
  - Right-hand-side (RHS) uses abstractions, tokens, and lexemes to define the LHS
  - Example says that `<assign>` (abstraction) is defined as a `<var>` (abstraction) followed by an equal sign (lexeme) followed by an `<expression>` (abstraction)

BNF Basics: Terminals & Non-Terminals



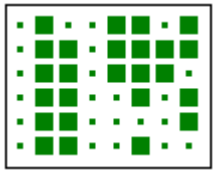
- Every abstraction must be defined and eventually resolve into lexemes and tokens
- Abstractions are known as **non-terminal symbols** (non-terminals)
  - They require further definition
- Lexemes and tokens are known as **terminal symbols** (terminals)
  - They do not require further definition

BNF Basics: Multiple Definitions



- Abstractions may have multiple definitions
  - Indicates that the language allows two (or more) syntactic forms for the abstraction

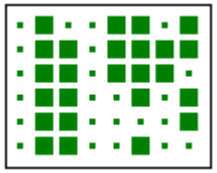
Context Free Grammar: Four-Tuple



- (T, N, S, P) where
  - T is the set of terminal symbols
  - N is the set of non-terminal symbols
  - S is the start symbol (a non-terminal)
  - P is the set of productions (definitions, rules) of the form:

    $A \rightarrow X_1 \ldots X_m$ where
    - A is an element of N
    - X_i is an element of $T \cup N$ where $1 <= i <= m, m>0$

BNF Example: Java's `if` Statement 1/3



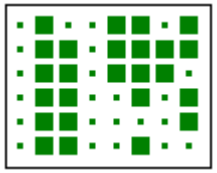
- A possible BNF definition for Java's if statement

  `<if_stmt> → **if (**<logic_expr>**)** <stmt>`

  `<if_stmt> → **if (**<logic_expr>**)** <stmt> **else** <stmt>`

- Which are BNF language symbols, abstractions, or lexemes?
- Which become part of the resulting syntax definition?
- Which become part of the resulting programming language (e.g. the code)?

BNF Example: Java's `if` Statement 2/3

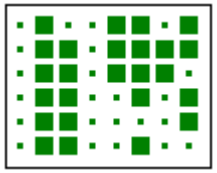


- Same `if` statement grammar rules using tokens in place of lexemes

```
<if_stmt> → IF_KEYWORD OPAREN
            <logic_expr> CPAREN <stmt>

<if_stmt> → IF_KEYWORD OPAREN
            <logic_expr> CPAREN <stmt>
            ELSE_KEYWORD <stmt>
```

BNF Example: Java's `if` Statement 3/3

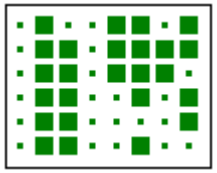


- An alternative BNF definition for `if` could be

```
<if_stmt> → if (<logic_expr>) <stmt>
          |  if (<logic_expr>) <stmt> else <stmt>
```

- The pipe symbol (|) precedes another RHS for the previous LHS
- Change the lexemes to tokens, what is the result?

BNF Supports Defining Lists of Values



- Lists are easy to define using BNF
  - For example, a list of identifiers requiring commas in between, such as declaring variables in Java

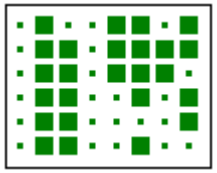
```
int x, y, z;
```

```
<declaration> → <datatype> <ident_list> ;

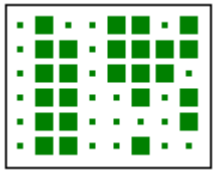
<ident_list> →<identifier>
             | <identifier>, <ident_list>
```

- Using *recursion* to describe syntax for a list

Create BNF for Java Syntax



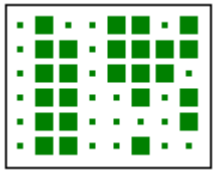
- Consider lexeme and token definitions
- Create BNF for the following:
  - While statement and class definition – Team Best
  - method definition and method call – Giraffes



Deriving Syntax from Grammars

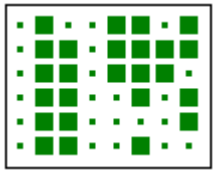
(Generating Statements)

Terminology



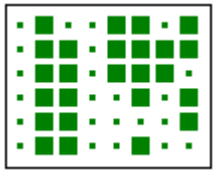
- Much research about grammars relates to natural language, we often see terminology used that is generic in nature
- We are focused on programming language statements and programs
  - Common to speak of **sentence generation** in this field
    - A *sentence* can be viewed as a statement, block of code or program, depending on context

Can Syntax Be Generated from BNF?



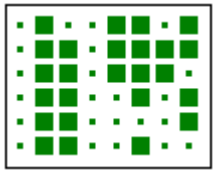
- We introduced BNF as a way to support language generation
- The process of generating a legal sentence (statement, program…) from BNF (grammar definition) is called **derivation**

Derivation (Generating Statements)



- We start with a high level **abstraction** (start symbol)
- We work through each **non-terminal**, choosing matching definitions until only **terminals** remain
  - Syntactically valid sentence
  - There is little chance that a generated program or statement will “work”

Grammar for Simple Language



(page 116 in Sebesta book)

Start Symbol

```
<program> → begin <stmt_list> end

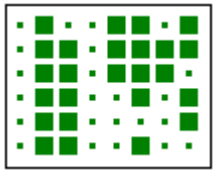
<stmt_list> →   <stmt>
              |  <stmt>; <stmt_list>

<stmt> → <var> = <expression>

<var> → A | B | C

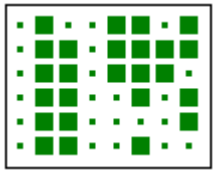
<expression> →  <var> + <var>
              |  <var> - <var>
              |  <var>
```

Process for Generating a Sentence 1/2



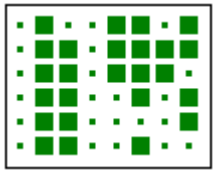
- We formalize the process of deriving a sentence from a grammar
  - Resolve the leftmost or rightmost non-terminal in the derivation
    - Defined as **leftmost** or **rightmost derivation**

Process for Generating a Sentence 2/2



- Each step in the derivation process results in a **sentential form** made up of terminals and, in all but the last derivation, non-terminals
  - The **final sentential form**, which contains only terminals, is the *generated sentence*
    - A possible statement, block, or program in the programming language described by the grammar
      - How many are possible?
      - How many are useful?

Generate a Program: Derivation



Use leftmost derivation to derive: `begin A=B+C; B=C end`

Start Symbol

```
<program> → begin <stmt_list> end

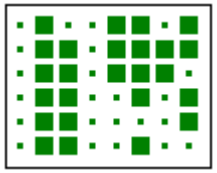
<stmt_list> →   <stmt>
              | <stmt>; <stmt_list>

<stmt> → <var> = <expression>

<var> → A | B | C

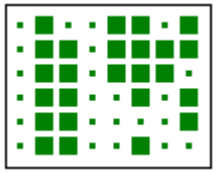
<expression> →  <var> + <var>
              | <var> - <var>
              | <var>
```

Generating All Programs is Infeasable



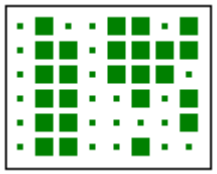
- By using the derivation process we could continually choose different subsets of non-terminals and terminals in order to generate all possible sentences (programs)
  - Set of possible programs is infinite for many syntax definitions; infeasible in finite time
- The goal of language generation is to help people see how to build a program through understanding the syntactic structures
  - Applying the generation process as part of designing and coding

Derive Program Code for Our BNF



- Try leftmost and rightmost derivation; does it change the result?
  - class definition
  - variable declaration (including initialization)
  - function definition
  - `while` statement
  - function call
  - `for` statement

Grammar for Simple Language



(page 117 in Sebesta book)

Start Symbol

<assign> → <id> = <expr>

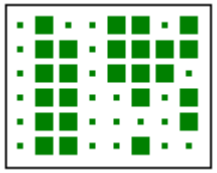
<id> → A | B | C

<expr> → <id> + <expr>
| <id> * <expr>
| (<expr>)
| <id>

Use leftmost and then rightmost derivation to derive the statement

A = B * (A + C)

Homework



- Sunday (10/4)
  - Journal entry for week 4
    - **2-1 Rule** (minimum 2 paragraphs, 1 quiz question)
- Monday (10/5)
  - Lab 2 Due: Flex & Make (Team and Individual)
- Wednesday (10/7):
  - Compilers book Appendix B

